

Java Section 1

INTRODUCTION TO JAVA APPLICATIONS

You will learn to write basic programs that display text, input numbers and do basic comparison.

Java Section 1

Introduction To Java Applications

1. **Developing Java** programs
2. **BlueJ** IDE
3. **Output (print) statements**
4. **Input statements** for numbers.
5. **Integer** primitive type
6. **Variables** and memory concepts
7. **Arithmetic operators**
8. **Basic decision making**
9. **Equality and relational operators**
10. **Getting Started**
11. **ASCII Character Set**
12. **Number Systems**

Lectures will emphasize the most important!

It is strongly recommended that you also read the recommending readings and complete the associated exercises.

JN

Java Editions and Versions

- **Java Editions: SE, EE, ME and Card**
 - **Java Standard Edition 23** (Java SE 23) - Current Version 23, 17Sep24 [History...](#)
 - Used for developing cross-platform, general-purpose applications.
 - Java is used in such a broad spectrum of applications that it has two other editions.
 - **Java Enterprise Edition (Java EE)**
 - Geared toward developing large-scale, distributed networking applications and web-based applications.
 - **Java Micro Edition (Java ME)**
 - geared toward developing applications for small, memory-constrained devices, such as BlackBerry smartphones, TV set-top boxes, printers etc
 - **Java Card**
 - Secure environment for applications to run on smart cards and trusted devices with limited memory and processing capabilities.
- **Java Programming Language**
 - Applies to all

Developing Java Programs

- A Computer Program [Software Program, Program] is:
 - A set of instructions given to a computer to perform a task
 - A Java program is referred to as a Java Application
 - Java programs are run/executed on a **Java Virtual Machine (JVM)**

Simplified Program Development Cycle

1. Edit

- Can use any text editor e.g. notepad to type in the code statements and store them in a file.

2. Compile

- javac** the command is the compiler used to compile Java programs
- Checks the syntax of code statements and generates *bytecodes* for execution

3. Execute

- java** the command to launch a Java application
- Executes the Java *bytecodes*

```
>notepad Welcome1.java
```

```
>javac Welcome1.java
```

```
>java Welcome1
```

```
Welcome to Java Programming!
```

BlueJ

Integrated Development Environment (IDE)

- Computer Programs are normally developed in an IDE.
- An IDE supports the cycle: 1. **Edit** -> 2. **Compile** -> 3. **Execute**

The screenshot displays the BlueJ IDE interface. The main window, titled 'BlueJ: Ch2Project', shows a project view on the left with a 'Welcome1' class icon. A dashed green arrow points from the 'Welcome1' icon to a text box that says '1. To edit: Double Click Icon'. Another dashed green arrow points from the 'Compile' button to a text box that says '2. To compile: Click Compile Button'. A third dashed green arrow points from the 'Welcome1' icon to a text box that says '3. To execute: Right Mouse Click on Icon and select main()'. A separate window titled 'Welcome1 - Ch2Project' shows the source code for 'Welcome1.java'. The code is as follows:

```
// Fig. 2.1: Welcome1.java
// Text-printing program.

public class Welcome1
{
    // main method begins execution of Java application
    public static void main( String[] args )
    {
        System.out.println( "Welcome to Java Programming!" );
    } // end method main
} // end class Welcome1
```

A dashed box in the top right corner of the main window contains the text: 'Icon shown represents a Java class called **Welcome1**'.

HelloWorld

HelloWorld

Displays (prints) two lines of text

HelloWorld.java

```
// Print hello and welcome text
public class HelloWorld
{ // begin class HelloWorld
```

```
Hello World
Welcome to Java Programming!
```

```
/*
 * the execution of a Java application begins at the main method
 */
```

```
public static void main( String[] args )
{ // begin method main
```

```
    System.out.println( "Hello World" );
    System.out.println( "Welcome to Java Programming!" );
```

```
} // end method main
```

```
} // end class HelloWorld
```

HelloWorld

Explanation: Comments

- **Comments**

- Are used to document programs and annotate code.
- The goal is to improve readability.
- The Java compiler ignores all comments.
- Java code when well written is readable without the need for many comments.

- **Implementation Comments**

- **Single or End-of-line comment** begins with a `//`

The comment terminates at the end of the line on which it appears.

```
// begin class HelloWorld
```

- **Multi-line or Traditional comment**, delimited by `/* ... */`

All text between the delimiters is ignored, normally spread over several lines.

```
/*  
 * the execution of a Java ...  
 */
```

- **Documentation Comments**

- Known as **doc** comments are delimited by `/** ... */`
- They can be extracted to HTML files for web pages using **javadoc**

HelloWorld

Explanation: Blank lines, Declaring a class

- Blank lines
 - And spacing, are used to make a program more readable.
 - Blank lines, spaces, and tabs are white-space characters ignored by compiler
- Every Java program consists of at least one user-defined class

```
public class HelloWorld
```

- Begins the class declaration for class HelloWorld
- **Keywords** are words that are reserved for use by Java, always **lowercase**.
 - keyword **public** means this class is accessible to other classes
 - keyword **class** declares the class, followed by class name
- Naming classes: capitalize every word, as we cannot use spaces or punctuation.
 - Convention is **ClassName** known as **PascalCase** or **Upper camelCase**
- The name is known as an **identifier**, a series of characters consisting of letters, digits, underscores (`_`) and dollar signs (`$`). An identifier cannot start with a digit.
- Java is **case sensitive** (capitalization matters) - **Hello** and **hello** are different

HelloWorld

Explanation: Braces

```
public class HelloWorld
{ // begin class HelloWorld
  class code goes here
} // end class HelloWorld
```

- **Curly Braces**

- A left opening brace, aka curly bracket, {, begins the body of every class declaration.
- A corresponding right closing brace, }, must end each class declaration.
- Braces must occur in matching pairs { }
- Code between braces should be indented, by normally 3 or 4 spaces.
- Indentation improves readability especially for expert readers. **Just do it!**

- **Brace styles**

1. **Opening curly brace on line by itself** (as above).

- Corresponding braces are vertically aligned.
- **Emphasises each block/section**

2. **Opening curly brace at end of previous line.**

- The opening curly brace at end of the line.
- Reduces lines of code to view.

```
public class HelloWorld {
  public static void main( String[] args ) {
    System.out.println( "Hello World" );
  }
}
```

HelloWorld

Explanation: the main method

```
public static void main( String[] args )  
{ // begin method main  
  
    System.out.println( "Hello World" );  
    System.out.println( "Welcome to Java Programming!" );  
  
} // end method main
```

- Java class declarations normally have one or more methods.
 - Methods perform tasks and can return information on completion.
 - Each method must enclose its statements between **{ }**
 - Left brace **{** begins body of method declaration, Right brace **}** ends it.
- The **main** method is the starting point of every Java application.
 - Keyword **void** indicates that this method will not return any information.
 - **main** must be defined as **public static void main(String[] args)**
- Statements instruct Java to perform the requested action
 - **System.out.println** means print (*display*) the text given as an argument
 - **System.out** is the standard output object (the command window)
 - The string is delimited by " " is called **a character string** or **string literal**
 - Most statements end with a semicolon ;

PrintText

Displays (prints) lines of text

PrintText

PrintText.java

// Print lines of text using print and println

// System.out.println print and move to the start of the next line

// System.out.print print and remain on the same line

```
public class PrintText
{
    public static void main( String[] args )
    {
        System.out.println( "Hello World");

        System.out.print( "# Welcome to " );
        System.out.println( "Java Programming #" );

        // can use \n to insert a newline, and start at the next line
        System.out.print( "\n at the\n\nUniversity\n    of\n Limerick" );
    }
}
```

```
Hello World
# Welcome to Java Programming #

at the

University
  of
Limerick
```

- A backslash (\) starts an **escape character** which means the following character will do something special e.g. \n is a newline meaning start a new line.

Escape Sequences

- An **escape sequence** (code) is a character which escapes the normal sequence of output and signals an alternative interpretation of a series of characters.
- In **Java**, the **escape sequence** a character preceded by a backslash \

Escape Sequence	Name	Description
\n	New line	Move the cursor to the beginning of the next line.
\t	Horizontal tab	Move the cursor to the next horizontal tab stop. Often set at column positions 1, 9, 17, 25, 33...
\r	Carriage return	Move the cursor to the beginning of the current line.
\\	Backslash	Insert a backslash character \.
\"	Double quote	Insert a double quote ". Since strings are enclosed in " ", need \"
\'	Singe quote	Insert a single quote '.
\b	Backspace	Move the cursor one space backwards, deleting the character at that position.
\f	Form feed	Insert a formfeed character, to inform printer to eject the current page.

Print Formatted Text

PrintFormatted

Print lines of text using **printf** and formatters

PrintFormatted.java

// **printf** to print formatted where the first string contains
// characters to print as is and special format specifiers
// such as %s, a placeholder to display a String.
// There then follows arguments for each
// format specifier.
// %7s which means to display the string
// a field width of 7 characters

```
public class PrintFormatted
{
    public static void main( String[] args )
    {
        System.out.printf( "%s\n", "Hello World" );

        System.out.printf( "# %s #\n", "Welcome to Java Programming" );

        System.out.printf( "\n%7s\n\n", "at the" );

        System.out.printf( "%s\n%5s\n%9s\n", "University", "of", "Limerick" );
    }
}
```

```
Hello World
# Welcome to Java Programming #

at the

University
of
Limerick
```

Outputs the
same text
as PrintText

Exercise 1

Print text using various print methods

- Using the print methods, each in turn:
`System.out.println( ... )`
`System.out.print( ... )`
`System.out.printf( format_string, ... )`
- Display three words to say hello on sequential lines e.g.

Hello

Hola

Ciao

- Which do you prefer?
- Which is easiest to understand?

Input Process Output

- Many Systems follow the **Input, Process, Output** cycle.

Cash ATM: **Input**: Card/Pin#/Request, **Process**: Check Account/Cash, **Output**: Notes

- Most Java applications are similar.
- Basic Java **input** is from the keyboard.
 - We must tell Java what we expect e.g. an integer number.
 - Use a **Scanner** object to get the keys typed and convert to the requested type.
- Input information is stored in variables
 - Variables correspond to memory storage locations
 - Each variable has a unique symbolic name and type.

- Consider a program to add two numbers:

Pseudocode	
1. Request and input two numbers	Input
2. Sum the numbers	Process
3. Display the sum	Output

AddNumbers

AddNumbers

Request two numbers, add them and print the sum [AddNumbers.java](#)

```
public class AddNumbers
{
    public static void main( String[] args )
    {
        // create a Scanner object to get input from the keyboard
        Scanner keyboard = new Scanner( System.in );

        System.out.println("Program to request and add two integer numbers");

        int number1;           // variable to store first number
        int number2;           // variable to store second number
        int sum;               // variable to store sum of number1 and number2

        System.out.print( "Enter the first number: " );    // prompt user
        number1 = keyboard.nextInt();                      // read first number

        System.out.print( "Enter the second number: " );  // prompt user
        number2 = keyboard.nextInt();                      // read second number

        sum = number1 + number2; // add the numbers, then store in variable sum

        System.out.printf( "The sum is %d\n", sum );      // display the sum
    }
} // end class AddNumbers
```


AddNumbers

Execution and imports

```
Program to request and add two integer numbers  
Enter the first number: 10  
Enter the second number: 5  
The sum is 15
```

- **Integers** – are whole numbers, like 7, 0, -22 and 1024
- Programs remember numbers and other data in the computer's memory and access that data through program elements called **variables**.
- **import declaration**
 - Tells the compiler to locate and import a class that is used in this program.
 - Classes are grouped into **packages** – which are named groups of related classes
- **Java Application Programming Interface (Java API)**.
 - The entire set of packages and classes is known as the **Java API** or **Java class library**
 - You use **import** declarations to identify the predefined classes used in a Java program
- **java.lang** package
 - By default, all the **java.lang** package classes are imported in every Java program
 - **java.lang** is the only package in the Java API that does not require an import declaration.

AddNumbers

Variables and Variable declaration statements

```
int number1;           // variable to store first number
int number2;           // variable to store second number
int sum;               // variable to store sum of number1 and number2
```

- Declare variables **number1**, **number2** and **sum** to hold data of type **int**
 - An **int** can hold a value in the range **-2,147,483,648** to **+2,147,483,647** (-2^{31} to $2^{31} - 1$)
 - A variable is a location in the computer's memory where a value can be stored for use later in a program.
- Must be declared with a **name (identifier)** and a **type** before they can be used.
 - A variable's **name** enables the program to access the value of the variable in memory. The name can be any valid identifier.
 - A variable's **type** specifies the kind of information stored at that location in memory.
- The **int value** may not contain commas e.g. 1000000 is a million.
- Multiple variables can be declared per **type**, using a comma separated list of names.
int number1, number2, sum;
 - The Java convention is to declare one variable per line.

AddNumbers

Creating a Scanner object

```
// create a Scanner object to get input from the keyboard  
Scanner keyboard = new Scanner( System.in );
```

- Specifies the type (**Scanner**) and name (**keyboard**) of an object variable that will be used in this program to read values from the keyboard.
 - A **Scanner** enables a program to read data for use in a program.
 - Data can come from many sources, such as the keyboard or a file on disk.
 - Before using a **Scanner**, you must create it and specify the source of the data.
- The equals sign (=) in a variable declaration indicates that the variable will be initialized with the result of the expression to the right of the equals sign.
- The **new** keyword creates an object, in this case a **Scanner** object.
- **System.in** is the **Standard Input object**, created at the start of program execution to enable an application to read bytes of information entered by the user.
- The Scanner object provides methods such as **nextInt()** to convert these bytes into types, in this case **int**

AddNumbers

Input: Prompt and read a number

```
System.out.print( "Enter the first number: " );    // prompt user
number1 = keyboard.nextInt();                     // read first number
```

Prompt (Output)

- Prompt the user to enter a number using **System.out.print**
- Note the character string finishes with a **colon :** and a **space** for better user readability.
- **System** is a class, in a package of classes **java.lang** (automatically imported).

Read (Input)

- The **Scanner** method **nextInt** waits for, and reads, an integer from the keyboard.
 - The user must press the **Enter** key to *enter* the number typed.
 - When the Scanner has a valid number, it will return the value and it will be stored in the variable **number1** using the **= (the assignment operator)**.

Assignment

- The **= assignment operator** is used to assign a **value** to a **variable**.
- Only a variable identifier can occur on the left of an **= assignment operator**.
- An **expression** can be on the right of an **=**, and it is evaluated before assigning.

AddNumbers

Process and Output

```
sum = number1 + number2; // add the numbers, then store in variable sum

System.out.printf( "The sum is %d\n", sum );           // display the sum
```

Assignment expression

- Calculations are done using algebraic expressions and the result is generally stored in a **variable** for later retrieval.
- Above, the **sum** variable is assigned the value of **number1 + number2**
 - Use an expression **number1 + number2** to add the numbers.
 - Complete the **assignment** to the variable **sum** using the **assignment operator (=)**.
 - The expression becomes a statement by appending the **;**

Formatted Output

- **%d** in the **printf** format string is a format specifier (placeholder) for an integer value, such as **int**.
- **%d** means **decimal integer**, display the corresponding **value** as a decimal (base 10)
- The value of **sum** will be printed where the **%d** format is placed in the format string.

Algorithms and Pseudocode

Exercise 2

An **algorithm** is:

- a specified procedure to solve a problem
- a sequence of definite and effective instructions, which terminates with the production of correct output from the given input.

A **computer program** is

essentially an elaborate algorithm that goes through a series of steps to arrive at a specific outcome [Intel].

Pseudocode

is an informal language that allows programmers outline and develop algorithms or solutions to problems.

- Pseudocode is generally a sequence of steps, optionally numbered.
- Often easier to first write pseudocode to outline a possible solution before coding.

Exercise 2

Write pseudocode (a sequence of numbered steps) for the following:

Prompt the user to enter two numbers and display the largest.

Exercise 2: Pseudocode

Prompt the user to enter two numbers and display the largest.

1. Display introductory text to inform user what program does
Display "Program to compare two numbers and output largest"
2. Prompt user to enter 1st number,
Display "Enter 1st number: "
3. Input and store 1st number
4. Prompt user to enter 2nd number,
Display "Enter 2nd number: "
5. Input and store 2nd number
6. Display "The largest number is: "
7. Compare Numbers and Display largest
If 1st number is greater than 2nd number
then Display the 1st number
else Display the 2nd number

Exercise 3

Square a number

- **Translate the following pseudocode into a Java class:**
 1. Declare a class NumberSquared
 2. Declare the main method
 3. Input an integer number
 4. Output the square of the number
- **Answer the following questions:**
 1. What will the full name of the source code file be?
 2. What **command** is used to compile this file into Java bytecodes?
 3. What is the resultant class file name?
 4. What **command** is used to load and execute this file?

Exercise 3

Program Code to Square a Number

```
import java.util.Scanner; // will use class Scanner for input

// 1. Declare a class NumberSquared
public class NumberSquared
{
    // 2. Declare the main method
    public static void main( String args[ ] )
    {
        // 3. Input and Store an integer number in a variable number
        Scanner keyboard = new Scanner( System.in );

        System.out.print( "Enter integer to square: " );
        int number = keyboard.nextInt();

        // 4. Output the square of the number
        System.out.printf( "Square of %d is %d\n", number, number*number);
    }
}
```

Enter integer to square: 5
Square of 5 is 25

Exercise 3

Answers/Homework

1. What will the full name of the source code file be?
NumberSquared.java
2. What **command** is used to compile this file into Java bytecodes?
javac *NumberSquared.java*
3. What is the resultant class file name?
NumberSquared.class
4. What **command** is used to load and execute this file?
java *NumberSquared*

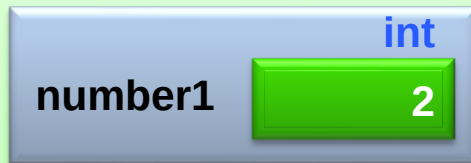
Homework

- **Consider how you would modify the code to check the input and ignore if 0**
 - Check if the number is not 0 before outputting its square.
 - Otherwise print a message informing the user that they have entered 0.

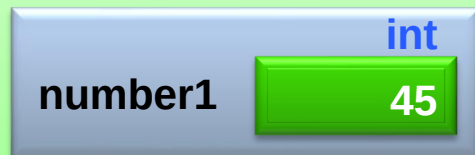
Memory Concepts - Variables

- Every **variable** has a **name**, a **type**, a **size** and a **value**
 - The **name** (**Java identifier**) corresponds to location in memory
 - A variable **name** should be meaningful (self-documenting)
 - A variable **name** by convention starts with lowercase and adopts **camelCase**.
 - When a new **value** is placed into a variable, it replaces the previous **value**
 - Reading the **value** of a **variable** from memory does not change the **value**
- Use a separate line to declare each variable

```
int number1;    // Declare variable of type int
number1 = 2;    // Assign value 2 to variable
```



```
number1 = input.nextInt(); // assume 45 entered
```



Memory locations in class **AddNumbers** after finding the **sum** of **number1** and **number2**, where **10** and **5** were entered.



Arithmetic operators

- Arithmetic calculations are frequently used in programs
- A set of symbols are used to represent the most common operations.

Operation	Binary Operator	Algebraic expression	Java expression: assume variables x and y
Addition	+	$x + y$	<code>x + y</code>
Subtraction	-	$x - y$	<code>x - y</code>
Multiplication	*	$x y$	<code>x * y</code>
Division	/	$x \div y$	<code>x / y</code>
Remainder	%	$x \bmod y$	<code>x % y</code>

- Arithmetic expressions in Java must be written in straight-line form to facilitate entering the expression in a program using text characters.
 - Expressions such as “a divided by b” must be written as `a / b`
 - All **constants**, **variables** and **operators** appear in a straight line
 - Parentheses: `()` are used to group terms in Java expressions as in algebraic.
 - In nested `()`, the expression in the innermost set of `()` is evaluated first.
 - **Integer division** / truncates the remainder, hence: `8/5` is 1
 - **Remainder operator** % returns the remainder: `8%5` is 3

Operator precedence and associativity

- Operators Precedence and Associativity are two characteristics of operators that determine the evaluation order of sub-expressions in the absence of parenthesis.
 - **Operator precedence** determines which **operator** is performed first in an expression with more than one **operators** with different precedence.
 - Some arithmetic operators act before others e.g., * before +
 - **Operator associativity** is used to decide which operator to process next when operators of the same precedence occur within an expression.
 - Use **parentheses** when needed, and even **redundant parentheses** to make an expression clearer (as in algebra).

Operation(s)	Operator(s)	Precedence Order of Evaluation	Associativity
Multiplication Division Remainder	* / %	High	Left to Right
Addition Subtraction	+ -		Left to Right
Assignment	=	Lowest	Right to Left

Expression: Order of Evaluation

Quadratic Equation (a 2nd-degree polynomial).

Algebra Quadratic: $y = 3x^2 + 5x - 8, x = 4$

Java Expression statement: `int y = 3 * x * x + 5 * x - 8;`

Recall: the arithmetic `*/+-` are binary operators, which work on two operands at a time

`y = 3 * 4 * 4 + 5 * 4 - 8;` `*` has Precedence over `+`, `*` associative left to right

`3 * 4` $\Rightarrow$ 12

`y = 12 * 4 + 5 * 4 - 8;` *ditto*

`12 * 4` $\Rightarrow$ 48

`y = 48 + 5 * 4 - 8;` *ditto*

`5 * 4` $\Rightarrow$ 20

`y = 48 + 20 - 8;` `+` - have equal precedence, associative left to right: `+` is next

`48 + 20` $\Rightarrow$ 68

`y = 68 - 8;` `-` only

`68 - 8` $\Rightarrow$ 60

`y = 60;` `=` operator has lowest precedence, so now can assign value to `y`

```
{  
    int x = 4;  
    int y = 3 * x * x + 5 * x - 8;  
    System.out.println("Evaluation of y = 3x2 + 5x - 8");  
    System.out.printf("y = %d, x = %d\n", y, x);  
}
```

Java Code

```
Evaluation of y = 3x2 + 5x - 8  
y = 60, x = 4
```

Output

Comparing Numbers

Equality and Relational Operators

- The **equality** and **relational operators** are used to compare **values**
 - The **result** of the comparison is either **true** or **false**.
 - They are **binary** operators, so they must always have **two** operands (on left and on right)

Operation	Operator	Algebraic expression	Java expression: <i>assume variables x and y</i>
Equality equals	<code>==</code>	$x = y$	<code>x == y</code>
Equality not equals	<code>!=</code>	$x \neq y$	<code>x != y</code>
Relational less than	<code><</code>	$x < y$	<code>x < y</code>
Relational greater than	<code>></code>	$x > y$	<code>x > y</code>
Relational less than or equal to	<code><=</code>	$x \leq y$	<code>x <= y</code>
Relational greater than or equal to	<code>>=</code>	$x \geq y$	<code>x >= y</code>

- Both of the **equality operators** have the same level of precedence.
 - Lower precedence than that of the relational operators.
 - And associate from left to right.
- The **relational operators** all have the same level of precedence
 - And also associate from left to right.

CompareNumbers

Request two numbers and compare - Input Numbers [CompareNumbers.java](#)

```
/*  
 * Print "Program to request and add two numbers"  
 * Prompt for first number, input and store  
 * Prompt for second number, input and store  
 * Compare the numbers using equality and relational operators == != < > <= >=  
 */
```

```
import java.util.Scanner;
```

```
public class CompareNumbers  
{
```

```
    public static void main( String[] args )  
    {
```

```
        Scanner keyboard = new Scanner( System.in );
```

```
        System.out.print( "Enter first integer: " );
```

```
        int number1 = keyboard.nextInt();
```

```
        System.out.print( "Enter second integer: " );
```

```
        int number2 = keyboard.nextInt();
```

```
Enter first integer: 10  
Enter second integer: 5  
10 != 5    Not Equals  
10 > 5     Greater than  
10 >= 5    Greater than or equal
```


CompareNumbers

Request two numbers and compare - Compare

CompareNumbers.java

- Compare the numbers using equality and relational operators

```
if ( number1 == number2 )
    System.out.printf( "%d == %d    %s\n", number1, number2, "Equals" );

if ( number1 != number2 )
    System.out.printf( "%d != %d    %s\n", number1, number2, "Not Equals" );

if ( number1 < number2 )
    System.out.printf( "%d < %d    %s\n", number1, number2, "Less than" );

if ( number1 > number2 )
    System.out.printf( "%d > %d    %s\n", number1, number2, "Greater than" );

if ( number1 <= number2 )
    System.out.printf( "%d <= %d    %s\n", number1, number2, "Less than or equal" );

if ( number1 >= number2 )
    System.out.printf( "%d >= %d    %s\n", number1, number2, "Greater than or equal" );
}
```

CompareNumbers

Request two numbers and compare - Explanation

- A condition is an **expression** that can either be **true** or **false**.

if selection statement

**if (condition)
statement**

```
if ( number1 == 4 )
    System.out.println( number1 );

if ( number1 > 6 ) ...
```

```
if ( number1 != 10 )
    number1 = 10;

if ( number1 <= 20 ) ...
```

- If the result of the **condition** expression when evaluated is **true** the **if** body **statement** following the **condition** will be executed, otherwise it will be skipped.

```
if ( number1 == number2 )
    System.out.printf( "%d == %d    %s\n", number1, number2, "Equals" );
```

- The **System.out.printf** will only execute if **number1** is equal to **number2**
 - i.e. if the **value** stored in **number1** is the same as the **value** stored in **number2**
- The **if** expression can be followed by
 - a single statement, an empty statement **;** or a block of statements enclosed in **{ }**
- A logic error is to place a **;** after the **if (condition)** as it acts as an empty statement.

More details on the **if** statement will follow later.

Exercise 4:

Introduce Conditions

- **For NumberSquared, modify to check the input and ignore if 0**
 - Check if the number is not 0 before outputting its square.
 - Otherwise print a message that the user has entered 0.
- **Translate the following psuedocode into a Java class:**
 1. Declare a class **NumberSquaredNoZero**
 2. Declare the **main** method
 3. **Input** an integer number
 4. If number is not zero then **output** the square of the number
else **output** "I'm ignoring your request to square 0!"
- **Same questions as for Exercise 3**
 1. What will the full name of the source code file be?
 2. What **command** is used to compile this file into Java bytecodes?
 3. What is the resultant class file name?
 4. What **command** is used to load and execute this file?

Exercise 4:

Program Code

NumberSquaredNoZero.java

```
import java.util.Scanner; // will use class Scanner for input

// 1. Declare a class NumberSquaredNoZero
public class NumberSquaredNoZero
{
    // 2. Declare the main method
    public static void main( String args[ ] )
    {
        // 3. Input and Store an integer number in a variable
        Scanner keyboard = new Scanner( System.in );

        System.out.print( "Enter integer to square (no 0's please): " );
        int number = keyboard.nextInt();

        // 4. If not zero then output the square of the number
        //           else display "I'm ignoring your request to square 0!"
        if (number != 0)
            System.out.printf( "Square of %d is %d\n", number, number*number);
        else // Same as if (number== 0)
            System.out.println("I'm ignoring your request to square 0!");
    }
}
```

Enter integer to square (no 0's please): 0
I'm ignoring your request to square 0!

Exercise 4:

if .. else / Answers

```
// 4. If not zero then output the square of the number
//           else display "I'm ignoring your request to square 0!"
if (number != 0)
    System.out.printf( "Square of %d is %d\n", number, number*number);
else // Same as if (number== 0)
    System.out.println("I'm ignoring your request to square 0!");
```

- An **else** is used instead of the alternative **if (number==0)** try it!
- The **if/else** selection statement, *more details later!*
 if (condition)
 statement // to execute when the condition evaluates to **true**
 else
 statement // to execute when the condition evaluates to **false**
- If the **condition** expression evaluates to **true** the statement following the **if (condition)** will be executed, if the **condition** evaluates to **false** the statement following the **else** will be executed.

Answers for Design Exercise Questions

- | | |
|--|--|
| 1. What will the full name of the <u>source code</u> file be? | NumberSquaredNoZero.java |
| 2. What command is used to compile this file into Java bytecodes? | javac <i>NumberSquaredNoZero.java</i> |
| 3. What is the resultant <u>class file</u> name? | NumberSquaredNoZero.class |
| 4. What command is used to load and execute this file? | java <i>NumberSquaredNoZero</i> |

Operator precedence and associativity

Including Equality and Relational Operators

- **Precedence** order in which arithmetic operators are processed
- **Associativity** order in which arithmetic operators of equal precedence are processed
- Use **parentheses** to make an expression clearer (as in algebra).

Operation Type(s)	Operator(s)	Precedence Order of Evaluation	Associativity
Multiplicative	* / %	Highest	Left to Right
Additive	+ -		Left to Right
Relational	< > <= >=		Left to Right
Equality	== !=		Left to Right
Assignment	=	Lowest	Right to Left

Based on the above table, what is the value of the following expressions?

a) $5 + 4 * 9 - 3$

b) $10 \% 3 + 4$

c) $10 - 2 > 15/3 - 1$

d) $8 != 4*2$

Getting Started 1

Classes and the main method

- All java applications are made up of one or more classes.
- The **class identifier/name** should adopt **PascalCase** aka **Upper camelCase**
 - Normally descriptive identifiers are used to name a class
 - Begin with a capital letter, and for each subsequent word, begin with a capital letter
- Java is case sensitive. **HelloWorld** and **helloWorld** are distinct identifiers
- The public class must have a file name that is identical to the class name (plus the **.java** extension) in terms of both spelling and case.

*public class **HelloWorld** must be stored in file **HelloWorld.java***

- Normally, one of the classes must have a **main** method.
- To declare the **main** method, it must be as follows:

```
public static void main( String[] args )
```

Getting Started 2

Statements, Braces and Line numbers

Statements

- Programs are composed of statements.
- Each statement should begin on a new line.
- Each statement generally ends with a semi-colon ;
- Frequently programmers limit each line width to 80 characters.
- A long statement can be continued on subsequent lines.
 - Subsequent lines are normally indented.
 - Choose sensible places to break the statement.
 - To break **strings** use the *String concatenation operator* +
- The empty statement is a standalone ;

"A string to break"

↓

"A string " +

"to break"

Opening and Closing (left and right) curly braces

- { } are used to mark the beginning and end of sections of code.
- Its good practice to insert the { } as a pair to avoid syntax errors.

Line numbers

- Line numbers **are not a part of** java but are useful if the editor displays them.
- Error messages indicate the line number where the compiler discovered the error

Getting Started 3

Style and Indentation

Style

- The presentation of code is key to readability.
- It is important to observe the **style** of Java programs.
- Program styles vary but it is essential to be consistent.
- Use blank lines and spaces to enhance program readability.
- **Brace styles** Pick a Brace Style and be consistent.

Opening curly brace: 1. on a separate line

```
if (number > 0 )  
{ // on a separate line  
} // closing brace vertically aligned
```

2. At end of previous code line

```
if ( number > 0 ) { // at end of previous  
}
```

Indentation

- Indentation is used to make the structure of a program clearer.
- Indent statements within a class, a method declaration or a control statement
 - Indent the entire body of each class or method declaration one “level” of indentation between the { } that delimit the body of the class or method.
- The indent size varies, but 2 to 4 are widely used.
- The Tab key may be used to create indents, but tab stops may vary in editors.

Documentation and Javadoc comments

Introduction and File header comment

Java programs are normally documented within the source code files.

- **Documentation** comments are special comments delimited by **/**** and ***/**
 - The text between the **/**** and ***/** is ignored by the compiler.
 - The comments allow us to embed the code documentation with the code itself, for automatic extraction to more readable formats e.g. web pages.
 - The comments can contain special block tags such as **@author**, **@version**
- Every program should begin with a **file header comment**
 - The file header comment is the first comment in the source code file.
 - The **Javadoc** comment includes: the purpose of the program, author and version.

```
/**
 * Class to input and compare two numbers
 * @author Jo McNeill
 * @version 10 Nov 2020, 10:30
 */
```

- The **javadoc** utility program generates HTML documentation
 - An IDE will have a command to generate the **Javadoc** documentation

Documentation and Javadoc comments

method and variable comments

- Typical **Javadoc** documentation for a method
 - There are many block tags `@param` `@return` see [How to Write Doc Comments](#)

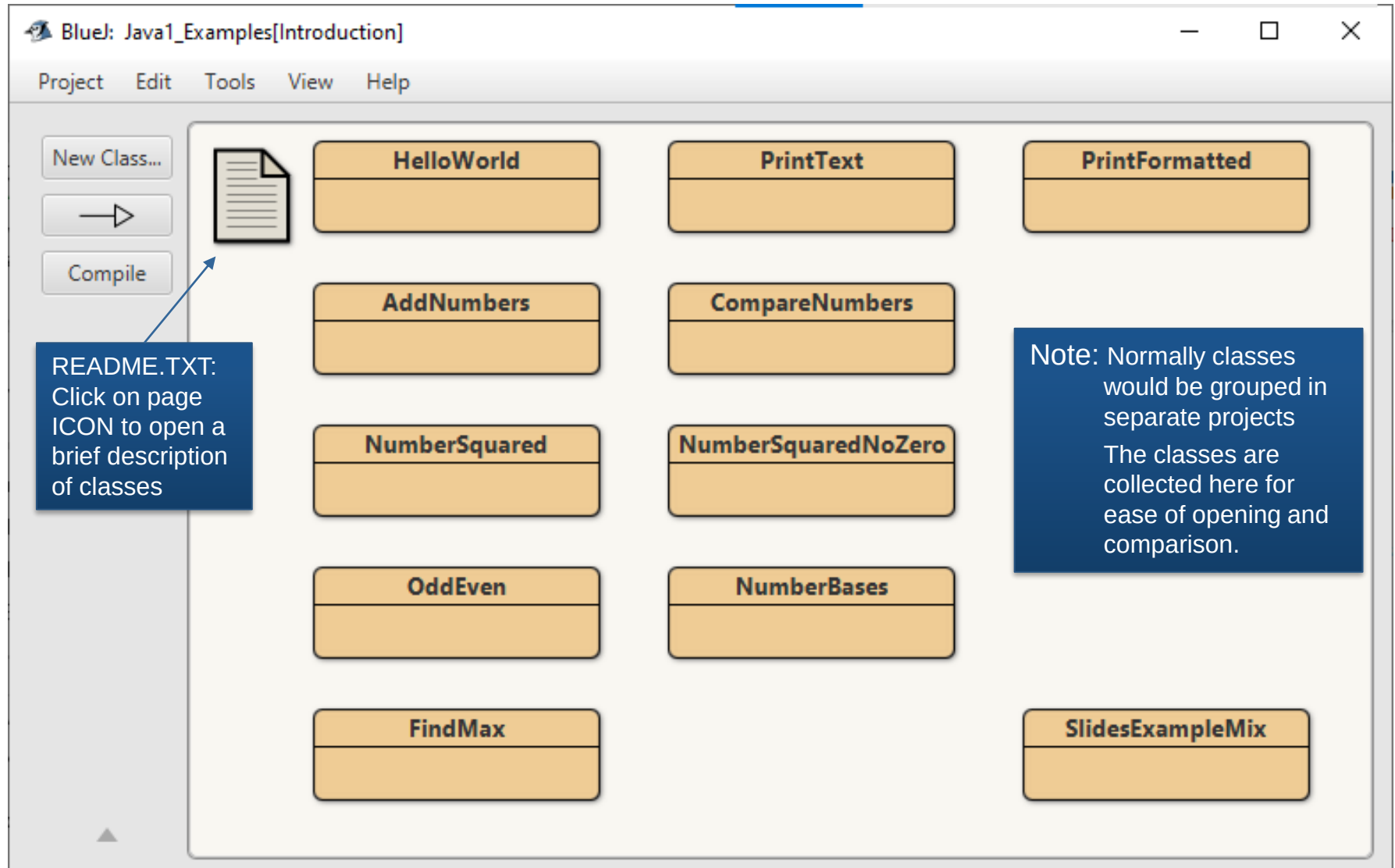
```
/**
 * Short one line description of what the method does.
 *
 * A optional longer (multi-line) description can be placed here .
 *
 * @param parameterName    Description text goes here (repeat for each parameter)
 * @return Description of what method returns ( if not of type void)
 */
```

- Typical **Javadoc** documentation for a variable
 - Not used for local variables.
 - Rarely used for instance variables (details later!).

```
/**
 * Description of the variable is placed here.
 */
private int variableName;
```

BlueJ: Java1_Examples

Examples discussed in this section: Go Explore!



BlueJ: Java1_Examples

Examples discussed in this section: Go Explore!

BlueJ: Java1_Example

Project Edit Tools View

New Class... → Compile

README.TXT: Click on page ICON to open a brief description

PROJECT TITLE:	Java1_Examples
PURPOSE OF PROJECT:	Examples of introductory Java Classes for input and output
VERSION or DATE:	AY2025
HOW TO START THIS PROJECT:	Right mouse click on a Class Icon and click on main + OK to execute
AUTHORS:	John
Output Examples	
HelloWorld	Print hello and welcome text using println
PrintText	Print lines of text using print and println, introduce \n
PrintFormatted	Print lines of text using printf and formatters
Input Examples	
AddNumbers	Request two numbers, add them and print the sum
CompareNumbers	Request two numbers, compare for equality and relationship
NumberSquared	Square a number
NumberSquaredNoZero	Square a number but check if zero first
OddEven	Determine if a number is odd or even
NumberBases	Demonstrate different number bases: Decimal, Octal, Hexadecimal.
Challenge #2	
FindMax	Find maximum of two integers (see Challenge description)
SlidesExampleMix	A wild mixture of examples from Java Introduction slides

FAQs for Challenge #2

From previous forum questions and lecture.

- What causes the error “**Cannot find symbol – class Scanner**”?
 - Ensure you **import java.util.Scanner;** before the class is declared.
- What causes the error “**variable minimum might not have been initialised**”?
 - **Local variables** are not initialised to a particular value when created.
 - However, before you can use a **local variable** in an expression, it must have been initialised.
 - If you have an **if ... else if** statement, consider an **if ... else if else** so that variable is guaranteed in the final **else** to be initialised.
 - You can initialise a **local variable** to a value such as 0 or -1 but need to check if this could end up as the answer in error when no statement in your **if ... else if** is executed.
- How many **if** statements are required?
 - Two if statements with simple conditions will suffice, but you will likely use more.
 - Once your solution works, and I deem the logic is correct, there is no penalty.
e.g. solutions with 10 if statements have received full marks!
 - Avoid looking for solutions, as the challenge is for you to reason out a solution.
- Do we have to complete the min calculation in both **main** and **min** methods.
 - Yes, see the requirements **c.** and **d.** of Challenge #2 document.

ASCII Character Set

Each character is represented by a number

	0	1	2	3	4	5	6	7	8	9
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	ff	cr	so	si	dle	dc1	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	"	#	\$	%	&	'
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	¥	]	^	_	'	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{		}	~	del		

- ASCII is a character encoding standard (1963) for 128 characters.
- Each character has an ASCII code.
- In the table, row and column digits combine to give the decimal equivalent number for the character
- Pick a character e.g. 'C'
 - ASCII Code is decimal 67
 - To find value for 'C'
 - a) Take row digit(s) to left i.e. 6
 - b) Take column digit on top i.e. 7
 - c) Combine row & column i.e. 67
- Find ASCII values for:
'A', 'a', '+', '1', '~'
- Find characters for:
48, 100, 0, 41

Number Systems

- A number **base** (aka **radix**)
 - denotes the number of digits used for counting e.g. *Base 10, Decimal, 10 digits*
- Number systems are positional
 - Each digit position in a number has a different positional value called a factor.
 - The contribution of the digit to the number value is the digit value multiplied by the factor.
 - Each factor, right to left, is a power of the base, starting at **base⁰** then **base¹** etc.

542_{10}	542 (decimal) is	$5 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$
542_{16}	0x542 (hexadecimal) is	$5 \times 16^2 + 4 \times 16^1 + 2 \times 16^0$
542_8	0542 (octal) is	$5 \times 8^2 + 4 \times 8^1 + 2 \times 8^0$

- Programmers can use different bases (radix) to represent numbers

<u>Number system</u>	<u>Base</u>	<u>Digits</u>	<u>Java Prefix</u>	<u>Java Examples</u>
Binary	2	0, 1	0b or 0B	0b11001, 0b11010, 0b11011
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	0	031, 032, 033
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	None	25, 26, 27
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F	0x or 0X	0x19, 0x1A, 0x1B

Number Bases/Equivalents

Decimal Base 10	Binary Base 2	Octal Base 8	Hexadecimal Base 16
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
...	...	...	...

- Home exercise, revise number base conversions...
- In particular:
 1. Converting from Decimal to Binary, Octal and Hexadecimal
 2. Converting from Binary, Octal or Hexadecimal to Decimal
- Why is it easier to represent Binary using Octal and Hexadecimal?

Base Number Conversion to / from Base₁₀ using *Positional Notation / Successive Division*

Converting a number N_B of base B to base 10 :

N_B will consist of digits d_i of base B
where each digit has a weight $0 \leq d_i < B$

Representing N_B as a sequence of digits
($d_k d_{k-1} \dots d_0$) _{B} then N_{10} in base 10 is

$$N_{10} = d_k B^k + d_{k-1} B^{k-1} + \dots + d_1 B^1 + d_0$$

Example convert to decimal:

$$N_{16} = 2AB_{16}, \text{ in Java notation } 0x2AB$$

$$\begin{aligned} N_{10} &= (2 \times 16^2 + A \times 16^1 + B \times 16^0)_{10} \\ &= 683_{10} \end{aligned}$$

where $A_{16} = 10_{10}$ $B_{16} = 11_{10}$ $C_{16} = 12_{10}$
 $D_{16} = 13_{10}$ $E_{16} = 14_{10}$ $F_{16} = 15_{10}$

Converting a number N from number base 10 to base B :

Consider each digit d_i of base B where $0 \leq d_i < B$,
 $/$ is integer division, **mod** is modulo (remainder)

$$d_k = (N / M^k) \bmod B$$

$$d_0 = N \bmod B, \quad d_1 = (N / M) \bmod B,$$

Example convert to hexadecimal :

$N = 683_{10}$, Widely used technique is iterative
division by the base & take remainders until quotient
becomes 0.

$$\begin{array}{r|l} 16 & 683 \\ \hline & 42 \quad B \\ 16 & 2 \quad A \\ \hline & 0 \quad 2 \end{array} \quad \uparrow$$

Read digits from
bottom to top
 $683_{10} = 2AB_{16}$

Demonstrate Numbers in Java

Decimal, Octal and Hexadecimal

```
public class NumberBases
{
    public static void main( String args[ ] )
    {
        Scanner keyboard = new Scanner( System.in );

        System.out.print( "Enter integer: " );
        int number = keyboard.nextInt();

        System.out.printf( "Number entered (in decimal) is %d\n", number );
        System.out.printf( "Number in octal (oct) is %o\n", number );
        System.out.printf( "Number in hexadecimal (hex) is %X\n", number );
        System.out.println();

        int num1 = 31;           // Decimal value is 31
        int num2 = 031;          // Decimal value is 25
        int num3 = 0x31;         // Decimal value is 49
        int num4 = 0b11111;      // Decimal value is 31

        System.out.println("Number values can be represented in decimal, hexadecimal, octal, binary");
        System.out.printf("In code, prefix: hex 0x or 0X, oct 0, binary 0b or 0B \n\n");

        System.out.println("Number values can be printed using printf in decimal, hexadecimal, or octal");
        System.out.printf("\nConsider: num1=%d, num2=%o, num3=0x%x, num4=0b11111\n", num1, num2, num3);

        System.out.printf("\t num1 = 31:      Dec %d, Oct %o, Hex %x\n", num1, num1, num1 );
        System.out.printf("\t num2 = 031:     Dec %d, Oct %o, Hex %x\n", num2, num2, num2 );
        System.out.printf("\t num3 = 0x31:    Dec %d, Oct %o, Hex %x\n", num3, num3, num3 );
        System.out.printf("\t num4 = 0b11111:  Dec %d, Oct %o, Hex %x\n", num4, num4, num4 );
    }
}
```

```
Enter integer: 17
Number entered (in decimal) is 17
Number in octal (oct) is 21
Number in hexadecimal (hex) is 11

Number values can be represented in decimal, hexadecimal, octal, binary
In code, prefix: hex 0x or 0X, oct 0, binary 0b or 0B

Number values can be printed using printf in decimal, hexadecimal, or octal

Consider: num1=31, num2=031, num3=0x31, num4=0b11111
num1 = 31:      Dec 31, Oct 37, Hex 1f
num2 = 031:     Dec 25, Oct 31, Hex 19
num3 = 0x31:    Dec 49, Oct 61, Hex 31
num4 = 0b11111: Dec 31, Oct 37, Hex 1f
```

JavaHTP, Deitel and Deitel

- The remaining slides are extracted from:
Deitel and Deitel, Java: How to Program, 9e
- Java Development Environment Overview
 - Discusses the five phases of developing a program
- Some generally good advice pop-outs
 - Good Programming Practices
 - Common Programming Errors
 - Error Prevention Tips
- Please review.
 - The first programming chapters, Chapters 2
 - freely available from the Pearson the publisher for older book editions are.
 - Number Systems, *see Appendix J in javahtp10e, H in 9e/8e*
 - ASCII Character Set, *see Appendix B in javahtp10e-8e*

Java Five phases: 1 Edit, 2 Compile, 3 Load, 4 Verify, 5 Execute

Edit, Compile, Load

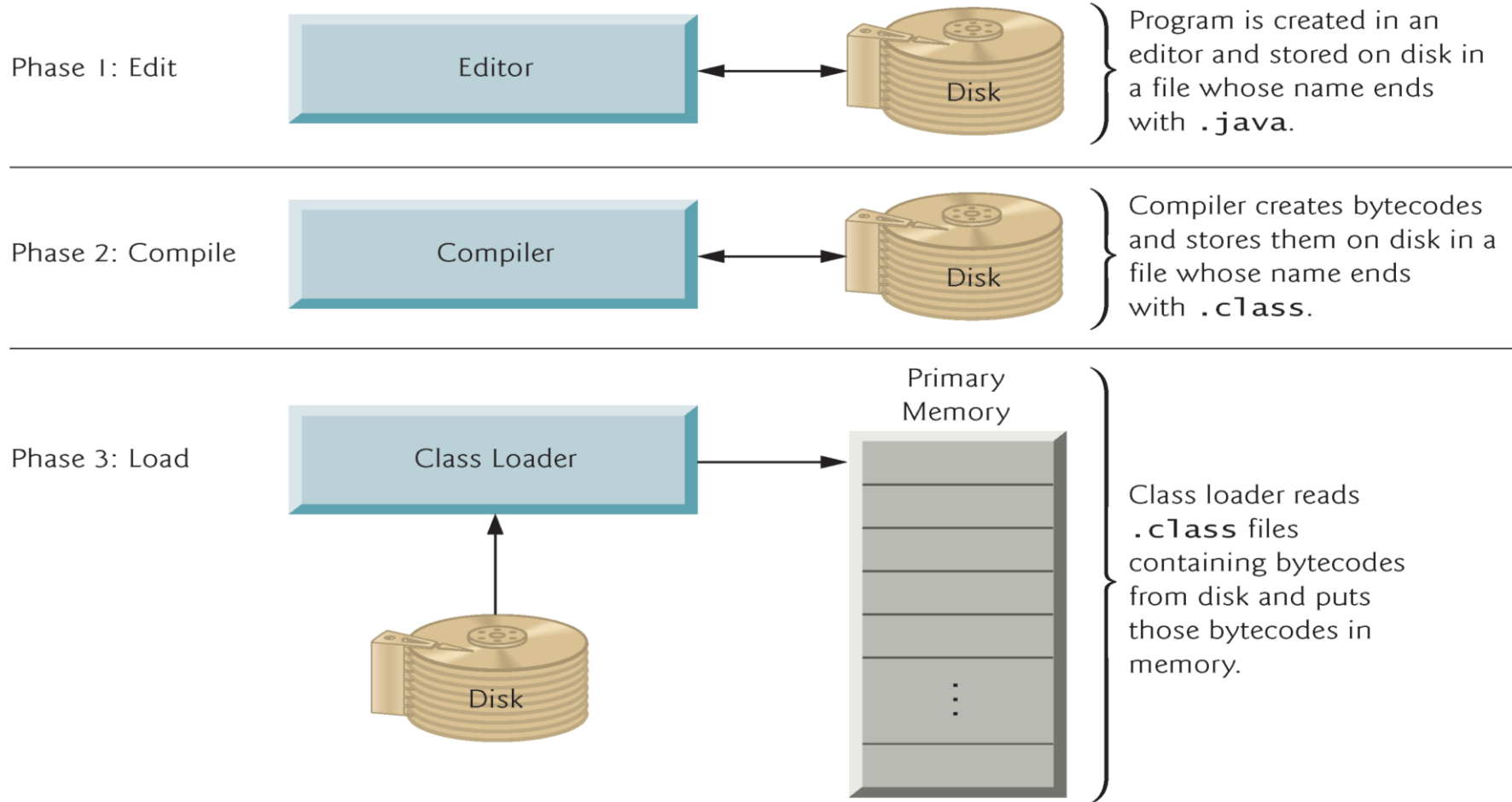


Fig. 1.1 | Typical Java development environment. (Part 1 of 2.)

► Java Five phases: 1 Edit, 2 Compile, 3 Load, 4 Verify, 5 Execute

Verify, Execute

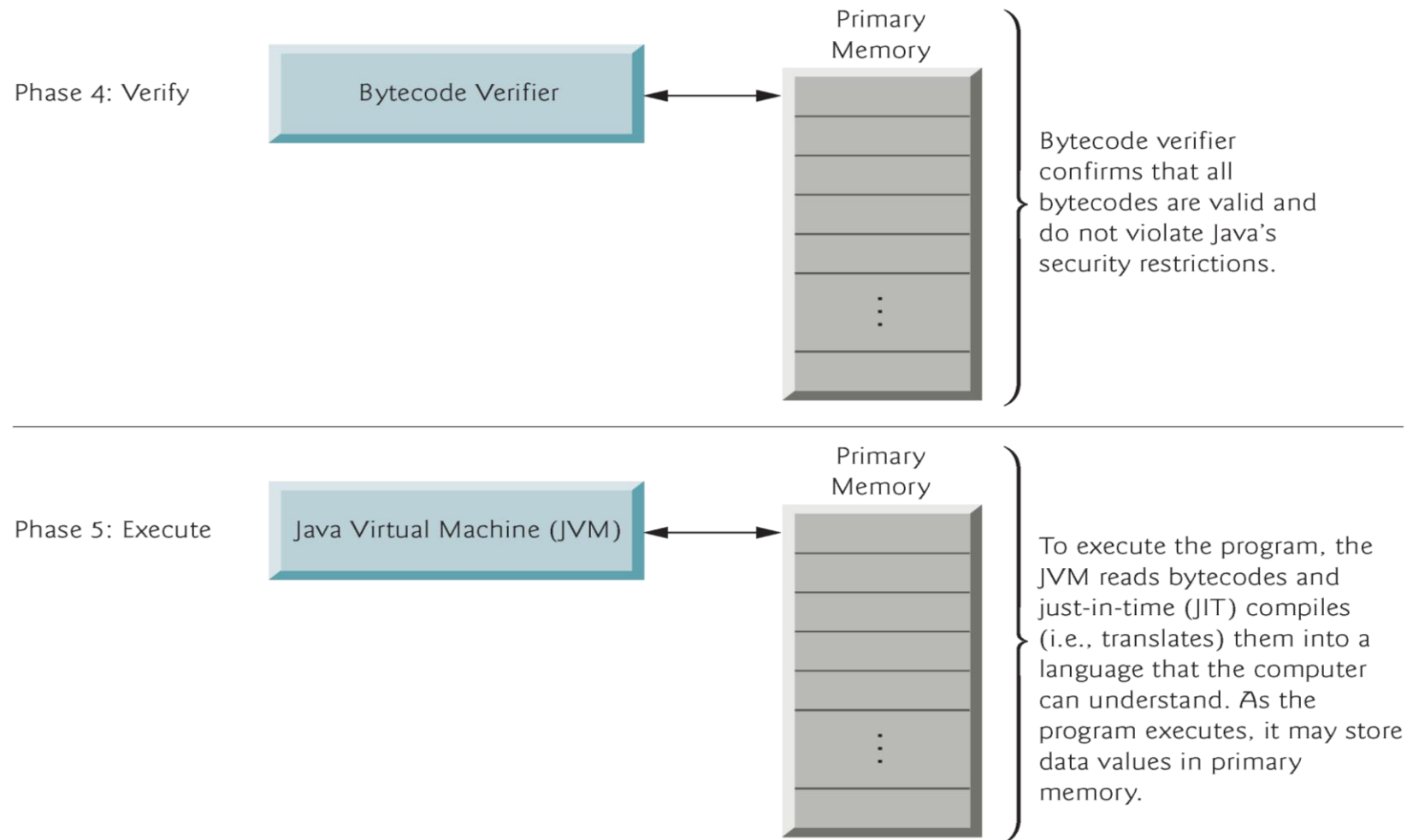


Fig. 1.1 | Typical Java development environment. (Part 2 of 2.)

Java and a Typical Java Development Environment

1 of 5

Phase 1 Edit: consists of editing a file

- Type a Java program (**source code**) using an editor.
- The source code is a sequence of characters.
- Make any necessary corrections.
- Save the program.
- A file name ending with the **.java extension** indicates that the file contains Java source code.

// A hello world application in Java

Hello.java

```
public class Hello
{
    public static void main( String args[] )
    {
        System.out.println("Hello world! Welcome to Java!");
    }
}
```

Java and a Typical Java Development Environment

2 of 5

Phase 2 Compile: Compiling a Java Program into Bytecodes

- The **Java compiler** translates Java source code into **bytecodes** that represent the tasks to execute.
 - Use the command **javac** (the **Java compiler**) to **compile** a program.
 - For example, to compile a program called **Hello.java**, you'd type

```
javac Hello.java
```

- If the program compiles with no errors, the compiler produces a **.class** file called **Hello.class** that contains the compiled version of the program in **bytecodes**.

Java and a Typical Java Development Environment

3 of 5

- **Bytecodes** are executed by the **Java Virtual Machine (JVM)**
 - **Virtual machine (VM)**: a software application that simulates a computer and hides the underlying operating system and hardware from the programs that interact with it.
- If the same **VM** is implemented on many computer platforms, applications that it executes can be used on all those platforms.
- **Bytecodes** are **platform independent** and are **portable**
 - They do not depend on a particular hardware platform.
 - The same **bytecodes** can execute on any platform containing a **JVM** that understands the version of Java in which the bytecodes were compiled.
- The **JVM** is invoked by the **java** command.
- For example, the command
java Hello
executes a Java application called **Hello**, in a file called **Hello.class**

Java and a Typical Java Development Environment

4 of 5

- **Phase 3 Load:** Loading a Program into Memory
 - The **Java Virtual Machine (JVM)** places the program in memory to execute it—this is known as **loading**.
 - **Class loader** takes the `.class` files containing the program's bytecodes and transfers them to primary memory.
 - Also loads any of the `.class` files provided by Java that your program uses.
 - The `.class` files can be loaded from a disk on your system or over a network.
- **Phase 4 Verify:** Bytecode Verification
 - As the classes are loaded, the **bytecode verifier** examines their bytecodes
 - Ensures that they're valid and do not violate Java's security restrictions.
 - Java enforces strong security to make sure that Java programs arriving over the network do not damage your files or your system (as computer viruses and worms might).

Java and a Typical Java Development Environment

5 of 5

- **Phase 5 Execute:** performs the execution of the program
 - The JVM **executes** the program's bytecodes.
 - JVMs typically execute bytecodes using a combination of interpretation and so-called **just-in-time (JIT) compilation**.
 - Analyzes the bytecodes as they're interpreted
 - A **just-in-time (JIT) compiler**—known as the **Java HotSpot compiler**—translates the bytecodes into the underlying computer's machine language.
 - When the JVM encounters these compiled parts again, the faster machine-language code executes.
 - Java programs go through *two compilation phases*
 - One in which source code is translated into bytecodes (for portability across JVMs on different computer platforms) and
 - A second in which, during execution, the bytecodes are translated into machine language for the actual computer on which the program executes.
- Errors caught during execution are known as **runtime errors**.
 - Two types **Fatal runtime errors** and **non fatal runtime errors**

Practices, Tips and Errors

1 of 6



Good Programming Practice 2.2

Use blank lines and spaces to enhance program readability.



Common Programming Error 2.1

*Forgetting one of the delimiters of a traditional or Javadoc comment is a syntax error. The **syntax** of a programming language specifies the rules for creating proper programs in that language, similar to how a natural language's grammar rules specify sentence structure. A **syntax error** occurs when the compiler encounters code that violates Java's language rules (i.e., its syntax). In this case, the compiler issues an error message and prevents your program from compiling. Syntax errors are also called **compiler errors**, **compile-time errors** or **compilation errors**, because the compiler detects them during the compilation phase.*

Practices, Tips and Errors

2 of 6



Common Programming Error 2.2

Java is case sensitive. Not using the proper uppercase and lowercase letters for an identifier normally causes a compilation error.



Common Programming Error 2.3

A `public` class must be placed in a file that has the same name as the class (in terms of both spelling and capitalization) plus the `.java` extension; otherwise, a compilation error occurs. For example, `public class Welcome` must be placed in a file named `Welcome.java`.



Error-Prevention Tip 2.1

When you type an opening left brace, `{`, immediately type the closing right brace, `}`, then reposition the cursor between the braces and indent to begin typing the body. This practice helps prevent errors due to missing braces. Many IDEs insert the braces for you.

Practices, Tips and Errors

3 of 6



Common Programming Error 2.5

When a semicolon is required to end a statement, omitting the semicolon is a syntax error.



Error-Prevention Tip 2.3

When the compiler reports a syntax error, the error may not be on the line indicated by the error message. First, check the line for which the error was reported. If that line does not contain syntax errors, check several preceding lines.



Good Programming Practice 2.6

Indent the entire body of each method declaration one “level” between the braces that define the body of the method. This makes the structure of the method stand out and makes the method declaration easier to read.



Good Programming Practice 2.7

To improve program readability, follow the closing brace of a method body or class declaration with a comment indicating the method or class declaration to which the brace belongs.

Practices, Tips and Errors

4 of 6



Error-Prevention Tip 2.2

When learning how to program, sometimes it's helpful to "break" a working program so you can familiarize yourself with the compiler's syntax-error messages. These messages do not always state the exact problem in the code. When you encounter such syntax-error messages, you'll have an idea of what caused the error. [Try removing a semicolon or brace from the program of Fig. 2.1, then recompile the program to see the error messages generated by the omission.]



Error-Prevention Tip 2.5

Each syntax-error message contains the file name and line number where the error occurred. For example, `Welcome1.java:6` indicates that an error occurred in the file `Welcome1.java` at line 6. The remainder of the error message provides information about the syntax error.

Practices, Tips and Errors

5 of 6



Common Programming Error 2.8

Forgetting the left and/or right parentheses for the condition in an `if` statement is a syntax error—the parentheses are required.



Common Programming Error 2.9

Confusing the equality operator, `==`, with the assignment operator, `=`, can cause a logic error or a syntax error. The equality operator should be read as “is equal to,” and the assignment operator should be read as “gets” or “gets the value of.” To avoid confusion, some people read the equality operator as “double equals” or “equals equals.”



Common Programming Error 2.10

Placing a semicolon immediately after the right parenthesis of the condition in an `if` statement is normally a logic error.

Practices, Tips and Errors

6 of 6



Good Programming Practice 2.14

Placing only one statement per line in a program enhances program readability.



Good Programming Practice 2.15

A lengthy statement can be spread over several lines. If a single statement must be split across lines, choose breaking points that make sense, such as after a comma in a comma-separated list, or after an operator in a lengthy expression. If a statement is split across two or more lines, indent all subsequent lines until the end of the statement.



Good Programming Practice 2.16

Refer to the operator precedence chart (Appendix A) when writing expressions containing many operators. Confirm that the operations in the expression are performed in the order you expect. If you are uncertain about the order of evaluation in a complex expression, use parentheses to force the order, exactly as you'd do in algebraic expressions.

Exercise 5: Homework

Write a program to convert Centigrade to Fahrenheit

- First write an expression that converts the value in a variable Centigrade to its equivalent value in Fahrenheit.
- Write out pseudocode for the steps that requests a user to enter the value to convert (as an integer) and displays the result as an integer.
- Now code, execute and debug the Java program. Test that the program works by ensuring that the Fahrenheit result is as expected.